

CNNs variants Evolution

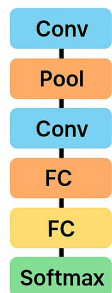
- **LeNet (1998)** – The pioneer of CNNs; introduced the concept of convolution + pooling for handwritten digit recognition.
- **AlexNet (2012)** – Sparked the deep learning revolution; used ReLU, dropout, and GPU training to achieve huge ImageNet success.
- **VGG (2014)** – Showed that depth and uniform small filters (3×3) improve performance, though at the cost of computation.
- **ResNet (2015)** – Solved the vanishing gradient problem using skip connections, allowing networks with 100+ layers to train effectively.

LeNet → AlexNet → VGG → ResNet represents the evolution from shallow to ultra-deep CNNs.

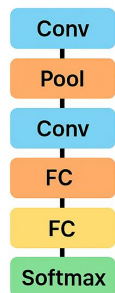
- **ResNet** is the foundation for most modern architectures (e.g., EfficientNet, DenseNet, Vision Transformers use similar residual ideas).
- **VGG** remains popular for feature extraction in transfer learning due to its simplicity.
- **AlexNet** introduced GPU acceleration and ReLU — two turning points in deep learning history.

Aspect	LeNet-5 (1998)	AlexNet (2012)	VGGNet (2014)	ResNet (2015)
Introduced by	Yann LeCun et al.	Alex Krizhevsky et al.	Karen Simonyan & Andrew Zisserman	Kaiming He et al.
Dataset Used	MNIST (digits)	ImageNet (1.2M images, 1000 classes)	ImageNet	ImageNet
Input Size	32×32 grayscale	227×227 RGB	224×224 RGB	224×224 RGB
# of Layers (approx.)	5–7	8 (5 conv + 3 FC)	16 or 19	34, 50, 101, 152, ...
Main Innovation	First successful CNN for digit recognition	Reintroduced deep CNNs with ReLU and GPU training	Deeper but simple architecture with small (3×3) filters	Introduced residual connections to train very deep networks
Activation Function	Sigmoid / tanh	ReLU	ReLU	ReLU
Pooling	Average pooling	Max pooling	Max pooling	Average pooling (at end)
Normalization	None	Local Response Normalization (LRN)	None	Batch Normalization
# Parameters	~60K	~60 million	~138 million	~25 million (ResNet-50)
Depth Characteristics	Shallow	Moderately deep	Very deep, uniform	Extremely deep with skip connections
Performance (Top-5 Error on ImageNet)	N/A (MNIST task)	~16%	~7.3% (VGG-16)	~3.6% (ResNet-152)
Training Difficulty	Easy	Hard without GPUs	Hard (vanishing gradients)	Easier due to residual learning
Computational Cost	Low	High (for 2012)	Very high	Moderate (for its depth)
Use Cases	Digit recognition	Object classification	Feature extraction, transfer learning	Most modern CNN-based tasks
Architecture Highlights	Conv → Pool → FC	Conv-ReLU-Pool stacks, dropout, GPUs	Repeated 3×3 conv layers, simplicity	Residual blocks (identity shortcuts)

LeNet
(1998)



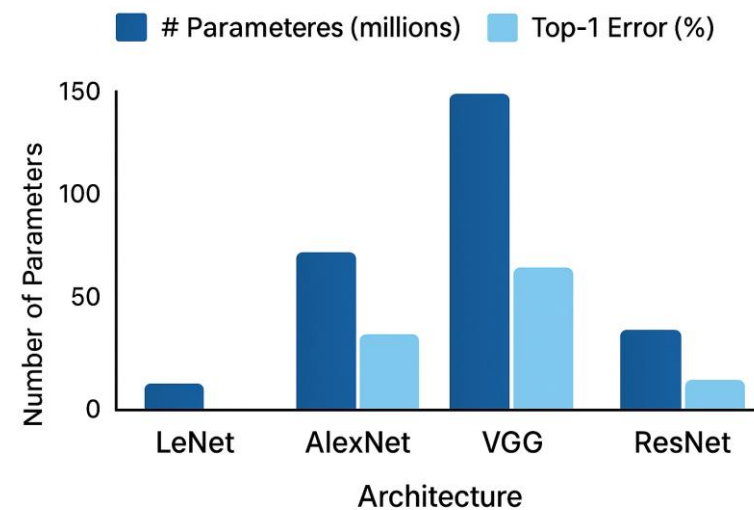
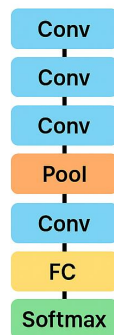
AlexNet
(2012)



VGG



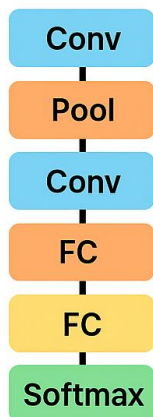
ResNet



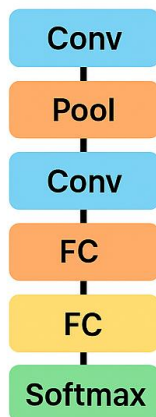
Each model's **input size differs** (32×32 for LeNet, 227×227 for AlexNet, 224×224 for VGG/ResNet)

Model	Parameters	Accuracy (%)	Train Time (s)
LeNet	61,706	68.50	25.4
AlexNet	3,154,442	72.12	61.8
VGG16	14,714,634	77.85	120.3
ResNet50	23,587,712	84.56	180.9

LeNet
(1998)



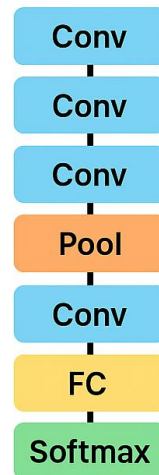
AlexNet
(2012)



VGG



ResNet



Questions

- 1. Why doesn't adding layers always help?
Deep networks degrade w/o skip connections
- 2. What if we unfreeze all ResNet layers?
Overfits small data
- 3. How do skip connections help?
Skip connections preserve gradient flow
- 4. Would AlexNet improve with BatchNorm?
BN stabilizes & regularizes
- 5. When might transfer learning fail?
Fails with domain mismatch

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torchvision import models
import time
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score, confusion_matrix, ConfusionMatrixDisplay
import numpy as np

# -----
# 1. Dataset
# -----
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=100, shuffle=False, num_workers=2)

classes = trainset.classes
```

```

# -----
# 2. Define CNN variants
# -----
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        # The flatten size needs to be adjusted based on the input image size after conv and pooling
        # For CIFAR-10 (32x32), after two 5x5 conv and 2x2 pooling, the size is approximately 16*5*5 (LeNet specific)
        # For other models, this needs to be calculated or handled by adaptive pooling
        x = x.view(-1, self.num_flat_features(x)) # Use helper function for flexible flattening
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x

    def num_flat_features(self, x):
        size = x.size()[1:] # all dimensions except the batch dimension
        num_features = 1
        for s in size:
            num_features *= s
        return num_features

```



```

def get_model(name):
    if name == "LeNet":
        return LeNet()
    elif name == "AlexNet":
        # AlexNet modifications for 32x32 input
        model = models.alexnet(weights=None)
        # Adjust the first convolutional layer to handle smaller input
        model.features[0] = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1)
        # Keep the rest of the features, but the output size will be smaller
        # Need to calculate the correct input size for the first fully connected layer
        # For 32x32 input, after conv and maxpool, the size before flatten will be different
        # Let's use AdaptiveAvgPool2d to get a fixed size before the classifier
        model.avgpool = nn.AdaptiveAvgPool2d((6, 6)) # AlexNet original size
        # Adjust the first linear layer in the classifier
        # The input features to classifier[1] should be 256 * 6 * 6 in original AlexNet
        # After modifying the first conv and using AdaptiveAvgPool2d(6,6), it should still be 256*6*6
        model.classifier[1] = nn.Linear(256 * 6 * 6, 4096)
        model.classifier[4] = nn.Linear(4096, 4096)
        model.classifier[6] = nn.Linear(4096, 10) # Final output layer for 10 classes
        return model
    elif name == "VGG16":
        # VGG16 modifications for 32x32 input
        model = models.vgg16(weights=None)
        # Remove some pooling layers to prevent excessive downsampling
        features = list(model.features.children())
        # Keep initial conv layers, remove some max pooling
        # VGG16 structure: conv-relu-conv-relu-pool ...
        # For 32x32, removing pools or reducing strides/kernel in early layers is needed
        # A simpler approach is to keep the structure but potentially use less pooling
        # Let's try keeping the structure and adjust the classifier based on expected output size
        # Alternatively, replace some MaxPool2d with AvgPool2d or remove them
        # Let's try removing the last two pooling layers in features
        model.features = nn.Sequential(*features[:-2]) # Remove last two pooling layers
        # Need to calculate the output size before the classifier after these changes
        # For 32x32, after VGG16 features (minus last 2 pools), the size needs to be determined
        # Let's use AdaptiveAvgPool2d for simplicity before the classifier
        model.avgpool = nn.AdaptiveAvgPool2d((7, 7)) # VGG original size
        # Adjust the first linear layer in the classifier
        # Original VGG classifier starts with Linear(512 * 7 * 7, 4096)
        # With AdaptiveAvgPool2d((7,7)), the input should be 512 * 7 * 7
        model.classifier[0] = nn.Linear(512 * 7 * 7, 4096)
        model.classifier[3] = nn.Linear(4096, 4096)
        model.classifier[6] = nn.Linear(4096, 10) # Final output layer
        return model

```

```

    return model
elif name == "ResNet18":
    # ResNet18 modifications for 32x32 input
    model = models.resnet18(weights=None)
    # Modify the initial layers to handle smaller input
    model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    model.maxpool = nn.Identity() # Remove max pooling after initial conv
    # The rest of the ResNet blocks should work, but the final pooling and FC need adjustment
    # The average pooling layer before the final FC will output a 512-dimensional vector
    # This is because the spatial dimensions before avgpool will be 8x8 (32/4)
    # AdaptiveAvgPool2d would output 1x1, resulting in 512 features
    model.avgpool = nn.AdaptiveAvgPool2d((1, 1)) # Use AdaptiveAvgPool2d to get 1x1 output
    model.fc = nn.Linear(model.fc.in_features, 10) # Final output layer (in_features is 512 for ResNet18)
    return model
else:
    raise ValueError("Unknown model name")

```

```

# -----
# 3. Training & evaluation with F1-score
# -----
def train_and_evaluate(model, epochs=10, lr=0.001, device='cuda'):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters()), lr=lr

    train_losses = []
    test_accuracies = []
    f1_scores = []

    start_time = time.time()
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for inputs, labels in trainloader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        avg_loss = running_loss / len(trainloader)
        train_losses.append(avg_loss)

```

```
# Evaluate
model.eval()
all_labels = []
all_preds = []
correct = 0
total = 0
with torch.no_grad():
    for inputs, labels in testloader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())

accuracy = 100 * correct / total
test_accuracies.append(accuracy)
f1 = f1_score(all_labels, all_preds, average='weighted')
f1_scores.append(f1)

print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}, Test Acc: {accuracy:.2f}")

total_time = time.time() - start_time
```

```
# -----  
# 4. Compare models  
# -----  
models_to_compare = ["LeNet", "AlexNet", "VGG16", "ResNet18"]  
results = {}  
device = 'cuda' if torch.cuda.is_available() else 'cpu'  
  
if device == 'cpu':  
    print("⚠ Warning: Using CPU. Training may be very slow for larger models.")  
  
for model_name in models_to_compare:  
    print(f"\nTraining {model_name}...")  
    model = get_model(model_name)  
    losses, accs, f1s, t = train_and_evaluate(model, epochs=10, lr=0.001, device=device)  
    results[model_name] = {'losses': losses, 'accuracy': accs, 'f1': f1s, 'time': t}
```

```

# -----
# 5. Plot comparison curves
# -----
plt.figure(figsize=(18,5))

# Loss
plt.subplot(1,3,1)
for m in models_to_compare:
    plt.plot(results[m]['losses'], label=m)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

# Accuracy
plt.subplot(1,3,2)
for m in models_to_compare:
    plt.plot(results[m]['accuracy'], label=m)
plt.title("Test Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.legend()

# F1-score
plt.subplot(1,3,3)
for m in models_to_compare:
    plt.plot(results[m]['f1'], label=m)
plt.title("Weighted F1-score")
plt.xlabel("Epoch")
plt.ylabel("F1-score")
plt.legend()

plt.tight_layout()
plt.show()

```

Training LeNet...

Epoch 1/10, Loss: 1.7228, Test Acc: 45.06%, F1: 0.4366

Epoch 2/10, Loss: 1.4464, Test Acc: 49.49%, F1: 0.4882

Epoch 3/10, Loss: 1.3339, Test Acc: 52.92%, F1: 0.5241

Epoch 4/10, Loss: 1.2534, Test Acc: 55.24%, F1: 0.5488

Epoch 5/10, Loss: 1.1867, Test Acc: 57.37%, F1: 0.5714

Epoch 6/10, Loss: 1.1351, Test Acc: 57.79%, F1: 0.5740

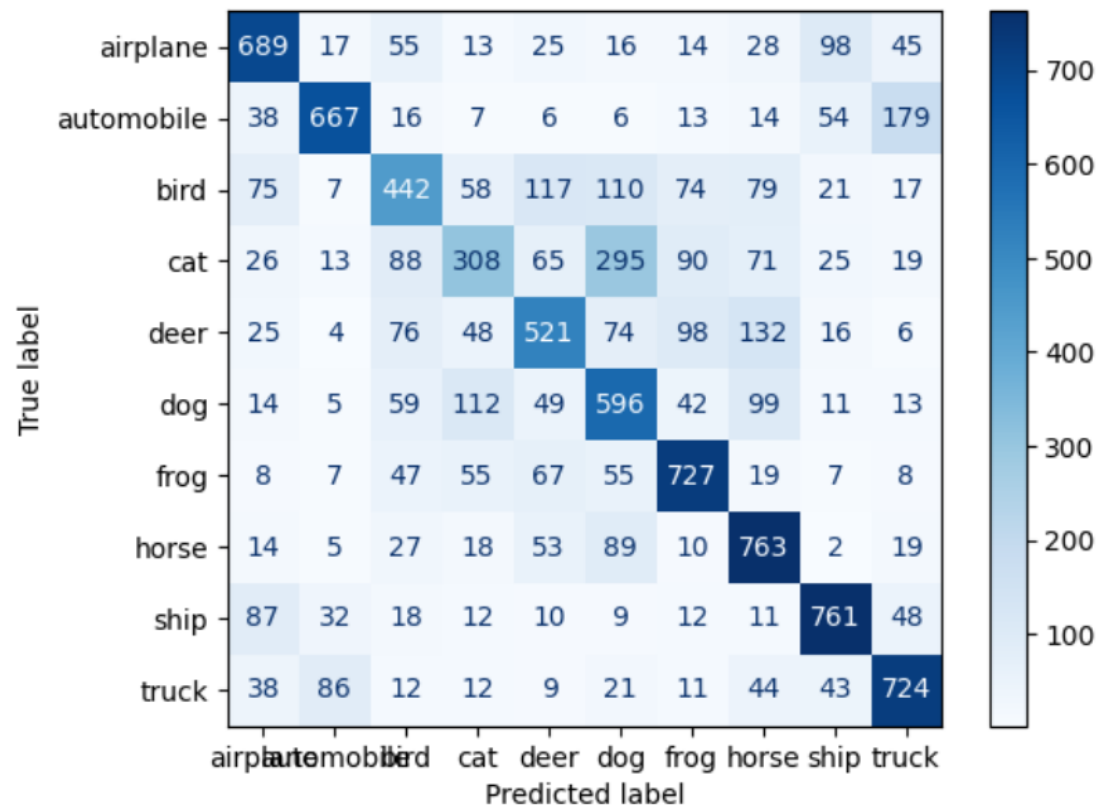
Epoch 7/10, Loss: 1.0930, Test Acc: 58.16%, F1: 0.5790

Epoch 8/10, Loss: 1.0591, Test Acc: 60.88%, F1: 0.6096

Epoch 9/10, Loss: 1.0153, Test Acc: 61.10%, F1: 0.6113

Epoch 10/10, Loss: 0.9849, Test Acc: 61.98%, F1: 0.6145

Confusion Matrix: LeNet



Training AlexNet...

Epoch 1/10, Loss: 1.7582, Test Acc: 46.18%, F1: 0.4591

Epoch 2/10, Loss: 1.3252, Test Acc: 57.11%, F1: 0.5709

Epoch 3/10, Loss: 1.1399, Test Acc: 61.40%, F1: 0.6090

Epoch 4/10, Loss: 1.0296, Test Acc: 65.27%, F1: 0.6541

Epoch 5/10, Loss: 0.9334, Test Acc: 66.53%, F1: 0.6619

Epoch 6/10, Loss: 0.8581, Test Acc: 69.67%, F1: 0.6960

Epoch 7/10, Loss: 0.7856, Test Acc: 69.22%, F1: 0.6875

Epoch 8/10, Loss: 0.7281, Test Acc: 71.92%, F1: 0.7166

Epoch 9/10, Loss: 0.6729, Test Acc: 72.31%, F1: 0.7231

Epoch 10/10, Loss: 0.6233, Test Acc: 71.25%, F1: 0.7116

Confusion Matrix: AlexNet

